



University of Asia Pacific
Department of Computer Science and
Engineering

Compiler Design
(CSE 430)

Submitted By

Mazharul Islam Sourav

ID: 22101133

Section: C-2

Semester: 4-2

Submitted To

Md Shafayatul Haque

Lecturer

Department of Computer Science
and Engineering

University of Asia Pacific

Work 04: Elimination of Left Recursion in a Grammar.

Code –

```
1  import os
2
3  def parse_block(lines):
4      g = {}
5      for line in lines:
6          l, r = line.split("->")
7          l = l.strip()
8          for p in r.split("|"):
9              p = p.strip()
10             if l not in g:
11                 g[l] = []
12                 g[l].append(p)
13     return g
14
15
16 def read_problems(file):
17     problems = []
18     block = []
19
20     with open(file, 'r', encoding='utf-8') as f:
21         for raw in f:
22             line = raw.strip()
23             if not line:
24                 if block:
25                     problems.append(parse_block(block))
26                     block = []
27                     continue
28             block.append(line)
29
30     if block:
31         problems.append(parse_block(block))
32
33     return problems
```

```

35 def eliminate_direct_left_recursion(g):
36     ng = {}
37     for nt in g:
38         a = []
39         b = []
40
41         for p in g[nt]:
42             if p.startswith(nt):
43                 a.append(p[len(nt):])
44             else:
45                 b.append(p)
46
47         if a:
48             nt2 = nt + ""
49             ng[nt] = []
50             ng[nt2] = []
51             for x in b:
52                 ng[nt].append(x + nt2)
53             for y in a:
54                 ng[nt2].append(y + nt2)
55             ng[nt2].append("ε")
56         else:
57             ng[nt] = g[nt]
58
59     return ng
60
61 def eliminate_indirect_left_recursion(g):
62     nts = list(g.keys())
63
64     for i in range(len(nts)):
65         ai = nts[i]
66
67         for j in range(i):
68             aj = nts[j]
69             np = []
70             for p in g[ai]:
71                 if p.startswith(aj):
72                     for x in g[aj]:
73                         np.append(x + p[len(aj):])
74                 else:
75                     np.append(p)
76             g[ai] = np
77
78     g = eliminate_direct_left_recursion(g)
79
80     return g

```

```
82 def print_block(g):
83     for nt in g:
84         out = " | ".join(g[nt])
85         print(f" {nt} -> {out}")
86
87
88 def print_problem_solution(i, original, transformed):
89     print("\n" + "=" * 56)
90     print(f"Problem {i:02d}")
91     print("-" * 56)
92     print("Input:")
93     print_block(original)
94     print("Solution:")
95     print_block(transformed)
96     print("=" * 56)
97
98
99 path = os.path.join(os.path.dirname(__file__), "Test_Case.txt")
100 problems = read_problems(path)
101
102 for i, g in enumerate(problems, start=1):
103     new_g = eliminate_indirect_left_recursion(g)
104     print_problem_solution(i, g, new_g)
```

Input -

1 $E \rightarrow E + T \mid T$

2

3 $T \rightarrow T * F \mid F$

4

5 $S \rightarrow x \mid y$

6 $A \rightarrow Sa \mid Bf \mid m$

7 $B \rightarrow Ab \mid Sc \mid n$

8

9 $A \rightarrow Aa \mid Ab \mid c \mid d$

Output -

Test Case 01 - (direct)

```
=====
Problem 01
-----
Input:
  E -> E + T | T
Solution:
  E -> TE'
  E' -> + TE' | ε
=====
```

Test Case 02 - (direct)

```
=====
Problem 02
-----
Input:
  T -> T * F | F
Solution:
  T -> FT'
  T' -> * FT' | ε
=====
```

Test Case 04 - (direct)

```
=====
Problem 04
-----
Input:
  A -> Aa | Ab | c | d
Solution:
  A -> cA' | dA'
  A' -> aA' | bA' | ε
=====
```

Test Case 03 – (Indirect)

```
=====
Problem 03
-----
Input:
  S -> x | y
  A -> Sa | Bf | m
  B -> Ab | Sc | n
Solution:
  S -> x | y
  A -> xa | ya | Bf | m
  B -> xabB' | yabB' | mbB' | xcB' | ycB' | nB'
  B' -> fbB' | ε
=====
```

Both input & output files are attached to submission and github repo as well.

Code Link : [GitHub](#)